

Monte Carlo Estimation of π on HTCondor

Understanding Throughput Using a Standardized Workflow

Snikitha Siddavatam

July 30, 2026

Contents

Background	1
Project Description	1
Project Scope	2
Project Challenge	2
Project Vision	2

Background

High-throughput computing (HTC) systems such as HTCondor let researchers split a large workload into many small, independent jobs that run in parallel across a shared pool of machines. The promise of HTC is throughput: the more jobs a user can move through the system per hour, the faster their science gets done. In practice, however, throughput depends on factors that are hard for users to see or reason about: how many jobs are submitted at once, how resource requests (CPU, memory, disk) affect where jobs can be matched, and how fair-share scheduling adjusts a user’s priority as they consume pool resources. Users routinely over-request memory “to be safe,” which shrinks the set of machines that can run their jobs and slows the whole workload down.

To study these effects systematically, we need a workload that is simple, embarrassingly parallel, and statistically well understood. Monte Carlo estimation of π fits perfectly: each job draws S uniform random points in the unit square and counts how many fall inside the quarter-circle ($x^2 + y^2 < 1$); the fraction converges to $\pi/4$, and the estimation error shrinks at the known rate of $1/\sqrt{N}$ in the total sample count N . Because every job is identical and independent, any variation in completion time reflects the behavior of the scheduling system itself rather than the workload.

Project Description

This project uses Monte Carlo estimation of π as a standardized benchmark workload for profiling job throughput on HTCondor. We submit J independent jobs, each generating a fixed $S = 10,000$ samples with a provably unique random seed derived from its cluster and process IDs, and accumulate results in

chronological order of job completion. From each run we measure how the absolute error of the π estimate decreases as the cumulative sample count N grows, and how throughput and per-job turnaround behave as J scales across five orders of magnitude: 10, 100, 1,000, 10,000, and 100,000 jobs. A pipeline of aggregation and plotting scripts cross-references each job's output file against the HTCCondor log, keeps only verifiably completed jobs, and produces convergence, runtime, turnaround, and cross-scale milestone comparisons.

Project Scope

The focus of this project is throughput characterization under controlled conditions: all jobs within an experiment request identical CPU, memory, disk, and clock-time limits, so differences in completion time reflect scheduling and pool variation rather than workload differences. We concentrate on (1) scaling behavior as the job count J grows from 10 to 100,000, (2) reproducible completion-time staggering via submit-side assignment of per-job sleep durations from a single seed chain, and (3) how throughput changes as the `request_memory` value is varied from 500 MB to 64 GB for an otherwise identical workload. We are *not* focusing on improving the Monte Carlo estimator itself (its $1/\sqrt{N}$ convergence is a known baseline used for validation), on optimizing single-job performance, or on modifying pool configuration or scheduler internals; the pool is treated as the system under observation, not something we tune.

Project Challenge

The central challenge is making runs at different scales genuinely comparable. Throughput on a shared pool is not stationary: submitting a large number of jobs lowers the user's fair-share priority relative to other users, so a 100,000-job run can experience different scheduling conditions than the 10-job run it is being compared against, and priority ideally needs to be reset between experiments. Correctness at scale is also nontrivial: with 100,000 jobs, a job must be counted as done only when both its output file exists at the Access Point *and* a matching `JOB_TERMINATED` entry appears in the HTCCondor log, and the aggregation pipeline must order tens of thousands of completions by timestamp without double-counting or dropping jobs. Finally, reproducibility requires that nothing about the experiment is left to chance inside the jobs: sampling seeds must be provably unique across clusters, and the sleep durations that stagger completions must be regenerable from a single starting seed rather than chosen independently by each job.

Project Vision

The goal is a reusable, fully reproducible benchmark workflow that turns vague intuitions about HTC throughput into measured curves: how fast jobs move through the system at each scale, when each 10% completion milestone is reached, and how much throughput is lost when memory requests are inflated. The impact is practical guidance for pool users (evidence for how to size job counts and resource requests so that workloads finish sooner) and a standardized instrument that facilitators can rerun at any time to observe how the pool's throughput behavior changes. Because the workload's statistical behavior is known exactly, the same workflow doubles as an end-to-end validation harness: if the error curve does not fall at $1/\sqrt{N}$, something in the pipeline, not the pool, is wrong.